

# Tutorial Sheet 2

## Memory Management

AUTHOR

Operating Systems Teaching and Research Unit

PUBLISHED

10.05.2026

## Introduction

### 1.1 The MMU

- a)** What are the tasks of memory management in operating systems?
- b)** What tasks does the MMU perform? What is the purpose of dividing a virtual address into segment number (or page number) and offset?
- c)** Is it possible that a virtual address is called for which no physical address exists? If so, how should the MMU handle it?
- d)** Why is it beneficial to implement the MMU as a hardware component rather than performing the necessary calculations in software?

### 1.2 System Performance

Given a demand paging system that is currently occupied as follows:

- CPU load: 20%
- ratio of page fault for accessing memory: 97,7%
- load of the other I/O devices: 5%

The aim is to optimize the CPU load in order to process all programs fast and efficiently. For each of the following suggestions, evaluate whether it would improve the CPU load (i.e. increase):

1. install a faster CPU
2. install a larger hard disk
3. increase the number of running/ready processes
4. reduce the number of running/ready processes
5. install more main memory
6. install a faster hard disk

## 2. Segmentation

For memory management using segmentation, the following segment table is given for a process:

| segment | base address | length |
|---------|--------------|--------|
| 0x0     | 0x528        | 320    |
| 0x1     | 0x232        | 58     |
| 0x2     | 0x136        | 120    |
| 0x3     | 0x2ee        | 60     |
| 0x4     | 0x33e        | 150    |
| 0x5     | 0x3de        | 110    |

Both the base address and the length are given in bytes.

- a) How many bytes are available to the process in physical memory?
- b) The address is coded on 16 bits. The maximum segment number is 16, how would you split the virtual address between the *segment number* and the *address* within the segment? What is the maximum segment length?
- c) The process requests the address 0x31 within segment number 2, what virtual address will be sent to the MMU?
- d) At what time is the virtual address from question c) determined?
- e) If two separate processes each have a memory segment labeled with the same segment number within their address space, how does the operating system ensure that these segments do not overlap in physical memory?
- f) The following virtual addresses are now requested by the process. What does the MMU return in each case?
  - 0x301e
  - 0x1012
  - 0x207a
  - 0x4095

### 3. Paging

Suppose we have a 32 bits system, managing virtual memory with paging. The page size is 4KiB and the TLB is empty.

The system uses a two level page table, the virtual addresses are divided as follows:

- 10 bits for the index of the first level page table

- 10 bits for the index of the second level page table
- 12 bits for the offset within the page

For instance, the address `0xc01b36e4` is divided like this: `0b1100000000|0110110011|011011100100`

- the index for the first level of the page table is 768
- the index for the second level is 435
- the offset is 1764

The Page Table Entry is composed of 20 bits for the page frame number, and the last 12 bits are used for metadata.

PTE

| 31...                 | ...12 | 11...                | ...3 | 2 | 1      | 0           |
|-----------------------|-------|----------------------|------|---|--------|-------------|
| Bits 31-12 of address |       | Other (D, A, PWT...) |      |   | U<br>S | R<br>W<br>P |

The last three bits represent the following flags:

- **U/S**: set to 1 if the page can be accessed either in user or supervisor mode
- **R/W**: set to 1 if the page is r/w, 0 if read-only
- **P**: set to 1 if the page is present in memory

Memory pages

|      |            |      |             |      |            |      |            |      |            |
|------|------------|------|-------------|------|------------|------|------------|------|------------|
| 1023 | 0x761c62fc | 1023 | 0xf0be9fcd  | 1023 | 0xe11d72a3 | 1023 | 0xbdd85d16 | 1023 | 0x0ed71b28 |
| 1022 | 0xec0fe000 | ...  | ...         | 1022 | 0x486143b2 | ...  | ...        | ...  | ...        |
| 1021 | 0x044e2fe1 | 261  | 0xf75c68ae  | 1021 | 0xcaa73b51 | 837  | 0x1384f7af | 979  | 0xe2931f71 |
| ...  | ...        | 262  | 0x5433c09f  | 1020 | 0x74b23643 | 836  | 0xcdb54af2 | 978  | 0xbf84f8f6 |
| 610  | 0x3ed733d1 | 263  | 0x53b40b50  | 1019 | 0x8c88cbd3 | 835  | 0x07959e6c | 977  | 0x212c5399 |
| 609  | 0x54826000 | 264  | 0xb3556864  | 1018 | 0x064a0623 | 834  | 0x45c0c51a | 976  | 0x1fa440ac |
| 608  | 0xb0b41068 | 265  | 0xd3e37a21  | 1017 | 0x5a7e0b13 | 833  | 0x13db28dd | 975  | 0xc8b8dda1 |
| 607  | 0x1432c000 | 266  | 0x5ce775f3  | 1016 | 0xb3ff3b53 | 832  | 0x0d72ddd7 | 974  | 0x73b5897a |
| ...  | ...        | 267  | 0xae71050   | 1015 | 0xd2da3be1 | 831  | 0x6eba6034 | 973  | 0x64c2d6f9 |
| 382  | 0xc13e4000 | 268  | 0x84ab8b74  | 1014 | 0x22abdd03 | ...  | ...        | 972  | 0xc6532b3c |
| 381  | 0x5fc46348 | 269  | 0x83cb8c1a  | ...  | ...        | 9    | 0x36556fd1 | 971  | 0xb0a368a1 |
| ...  | ...        | 270  | 0x79ae8204  | 674  | 0xa7f0a152 | 8    | 0x1fb73c5c | 970  | 0x910d0807 |
| 7    | 0x94ed9556 | 271  | 0x6b2ba370  | 673  | 0x5e55ba43 | 7    | 0xbcb8acb7 | 969  | 0xdf3f3877 |
| 6    | 0x771af000 | ...  | ...         | 672  | 0x747a7a3f | 6    | 0xb975c46e | 968  | 0x45dc0a39 |
| 5    | 0x554c9d87 | 5    | 0x3c3704fd  | 671  | 0x61ae7da2 | 5    | 0x4830d301 | 967  | 0x03b373e2 |
| 4    | 0x6cb4f481 | 4    | 0x73839ce3  | 670  | 0xf29c9732 | 4    | 0x59ff1322 | 966  | 0xb869f0bd |
| 3    | 0x0c9a4000 | 3    | 0xcce6399e0 | 669  | 0x05cbf3a1 | 3    | 0x5c6f1356 | ...  | ...        |
| 2    | 0xd9ffb83a | 2    | 0xd1fd6562  | ...  | ...        | 2    | 0x22da0265 | 2    | 0x322991db |
| 1    | 0xf0b77ac6 | 1    | 0xcd1fe587  | 1    | 0x6cd67cb1 | 1    | 0x65103266 | 1    | 0xc7051491 |
| 0    | 0x321e7b42 | 0    | 0xdd02d6e7  | 0    | 0x4215b193 | 0    | 0x3e797c52 | 0    | 0x58f86463 |

addr: `0xef2e5000`

addr: `0x771af000`

addr: `0x1432c000`

addr: `0x54826000`

addr: `0xc13e4000`

Suppose that a task is running, the page table pointer of that task contains `0xef2e5000`.

## Questions

1. The third page displayed above starts at the address `0x1432c000`. What is the associated page number?
2. The value `0x6cb4f481` is stored inside the first page, at the index 4. What is the address at which the value is stored?
3. What is the size of each page table level? Why is it convenient?
4. What is the total size of memory used by the page table for a single address translation?

# Task

The CPU is performing the following actions:

- 1. In user mode, read at the address 0x97ea0329
- 2. In user mode, write to the address 0x5fbc6582
- 3. In kernel mode, read at the address 0x5fcd6f0
- 4. In user mode, read at the address 0x5fbd2bad
- 5. In user mode, read at the address 0x97ffbc21
- 6. In user mode, read at the address 0x97ea0fe2

For each action, document the steps involved in MMU translation. Explain how the MMU accesses the Page Table Entry associated with the virtual address, translates the virtual address to a physical address, and handles any interrupts or exceptions encountered during the translation process.

TLB

| Translation Lookaside Buffer |            |       |
|------------------------------|------------|-------|
| Page                         | Page Frame | FLAGS |
|                              |            |       |
|                              |            |       |
|                              |            |       |
|                              |            |       |
|                              |            |       |
|                              |            |       |

Finally, update the TLB accordingly: each entry one after the other. To make things easier, don't flush the TLB and suppose that all entries are valid (!= present).

Note

On a more realistic system, the TLB contains more flags for caching etc.

The TLB is either flushed between each context switch or contains a **PCID** (Process Context Identifier) to associate each entry with a specific process.

## Bonus: Segmentation vs. Paging

Given a computer with **16 GiB** main memory and **64-bit** address space. You are considering whether to use segmentation or paging. You have the choice between 3 options.

Note: 1 GiB = 2<sup>30</sup> bytes, 1 MiB = 2<sup>20</sup> bytes

### Option 1: Paging with pages of 256 bytes in size

- 1.1 How many pages would there be in the page table to be able to translate any virtual address?
- 1.2 Why such a small page size could cause a lot of TLB misses.
- 1.3 What is the size of the page table for a single translation, with 1 level of page table?
- 1.4 What about 2 levels of page table?

**1.5** What about 7 levels of page table?

**1.6** Why is this number of levels problematic?

### **Option 2: Paging with pages of 256 MiB in size**

**2.1** How many entries would there be in the page table?

**2.2** Why is having pages of this size problematic?

### **Option 3: Segmentation with best fit as a strategy for searching for free memory**

**3.1** Is there any internal / external fragmentation?

### **Conclusion:**

Based on the questions discussed above, what can we conclude about the optimal solution for memory management?